SMARTAPPLY: A REAL-TIME COLLEGE ADMISSION AND TRACKING PLATFORM USING THE MERN STACK

A PROJECT REPORT

Submitted by

HANAA REHMAAN F (310624148028)

in partial fulfillment for the award of the Degree

of

BACHELOR OF ENGINEERING

in

DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)
2311CSE503R - FULL STACK DEVELOPMENT



EASWARI ENGINEERING COLLEGE,

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled **"SMARTAPPLY: A REAL-TIME COLLEGE ADMISSION AND TRACKING PLATFORM USING THE MERN STACK"** is the Bonafide work of **"HANAA REHMAAN F (310624148028)",** who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT
Dr. Justin Dhas Y

SUPERVISOR
Ms. Divyaanjali

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce-Examination

This is to certify that the Viva Voce Examination of the Project Work entitled **"SMARTAPPLY: A REAL-TIME COLLEGE ADMISSION AND TRACKING PLATFORM USING THE MERN STACK"** submitted by Mr./Ms. **HANAA REHMAAN F** (Reg. No. **310624148028**) in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** was conducted on **22-10-2026** (Date) in the Department of **Computer Science and Engineering (Artificial Intelligence and Machine Learning)**, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner	**External Examiner**
Name :	Name :
Designation :	Designation :
Address :	Address :

Date:

ABSTRACT

College admission in most institutions still depends on paper forms, repeated visits to the campus and status enquiries made by phone or in person. Students do not know where their application stands, admission officers spend time answering the same questions, and small data-entry mistakes in marks or missing certificates are discovered late, which delays the whole process.

SmartApply is a web-based college admission and tracking platform built on the MERN stack (MongoDB, Express.js, React.js and Node.js) that digitises this journey from application to enrolment. A student creates one digital profile, uploads documents once, applies to courses online and follows every stage of the application on a live timeline. When an admission officer changes a status or verifies a document, the change is pushed to the student's screen through Socket.IO without a page refresh.

Around this core workflow the system adds explainable decision-support modules: a Smart Application Assistant that flags mismatches in marks without rejecting the application, a Course Compatibility Engine, a What-If Simulator, a Scholarship Matcher and an administrative Fairness Audit. It also offers a global directory of over 10,000 universities filtered by region, one-step applications to several universities, a multilingual interface and a counsellor chatbot.

Security is provided by JSON Web Tokens, role-based access control and bcrypt password hashing, and the main flows were verified with automated tests. Every admission decision remains with a human officer.

Keywords: MERN stack, online admission, real-time tracking, Socket.IO, JWT authentication, decision support, fairness audit, chatbot.

ACKNOWLEDGEMENT

I express my sincere gratitude to the Principal **Dr. P. Deiva Sundari** and the Management of our college for providing me with the opportunity to carry out this project work.

I convey my heartfelt thanks to the **Head of the Department**, **Dr. Justin Dhas Y**, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), for the continuous encouragement and support.

I am extremely thankful to my **Project Guide**, **Ms. Divyaanjali**, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), for valuable guidance, suggestions, and motivation throughout the completion of this project.

I would also like to extend my gratitude to all the faculty members of the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning) for their support.

Finally, I thank my parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	4
	ACKNOWLEDGEMENT	5
1	INTRODUCTION	7
	1.1 Overview	7
	1.2 Problem Definition	8
	1.3 Objective	9
	1.4 Scope of the Project	10
	1.5 Motivation	11
2	RELATED WORK	12
	2.1 Literature Review	12
	2.2 Existing System	14
	2.3 Proposed System	15
3	SYSTEM ANALYSIS	16
	3.1 System Requirements	16
	3.2 Software and Hardware Requirements	18
4	SYSTEM DESIGN	19
	4.1 System Architecture	19
	4.2 Module Description	20
	4.4 Class Diagram	23
	4.5 Sequence Diagram	24
5	IMPLEMENTATION	25
	5.1 Implementation Overview	25
	5.2 Key Functions (Code Explanation)	27
6	CONCLUSION & FUTURE WORK	30
	6.1 Conclusion	30
	6.2 Future Enhancements	31
	APPENDIX – SOURCE CODE	32
	SCREENSHOTS OF THE PROJECT	43
	REFERENCES	48

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

Admission is the first formal contact between a student and a college, and it is also one of the busiest periods for the institution. Within a few weeks an admissions office must receive thousands of applications, check mark sheets and certificates, shortlist candidates, conduct interviews and allot seats. When this is done with paper forms and spreadsheets, the process is slow for the office and opaque for the student.

SmartApply is a full-stack web application that brings the whole admission journey onto one platform. It is developed with the MERN stack: MongoDB stores the data, Express.js and Node.js provide the REST API and the real-time server, and React.js provides the user interface. The system has three kinds of pages: public pages for visitors, a student portal and an administrator portal for admission officers.

A student builds a digital profile, uploads documents and applies to one or more courses. Each application then moves through ten defined stages, and every change made by an officer reaches the student instantly through a WebSocket connection.

The platform is designed for two groups of users with different needs. Students need clarity and guidance at every step, while admission officers need an organised workload and a reliable record of each decision. SmartApply serves both from a single database, so the information a student enters is exactly what the officer reviews.

1.2 PROBLEM DEFINITION

The conventional admission process has the following problems.

- **No visibility of status.** After submitting a form, the student has no way to know whether it is being verified, reviewed or kept on a waitlist, other than calling or visiting the college.
- **Repeated data entry.** The same personal and academic details are written again for every college and every course.
- **Late detection of errors.** A wrong percentage or a missing certificate is noticed only during manual verification, and the correction cycle costs days.
- **No guidance on course choice.** Students apply without knowing how well their marks, stream and interests fit a course or which scholarships they qualify for.
- **Limited reach.** Information about universities in other regions and countries is scattered, and portals are usually available in one language only.
- **No check on consistency of decisions.** The admissions office has no simple view of whether selection rates differ across streams or score groups.

The problem addressed by this project is therefore to design and implement a single web platform through which a student can apply online, track progress and receive updates in real time, and through which an admission officer can process applications with a complete audit trail.

These problems affect every party: students lose time and confidence, officers repeat the same work, and the institution has no reliable data with which to improve its process.

1.3 OBJECTIVE

The objectives of the project, and the part of the system that meets each one, are listed in Table 1.1.

No.	Objective	Achieved through
1	Digitise admission: register, build a reusable profile, upload documents and apply online	Profile, Document Locker, Application Wizard
2	Track every application in real time with instant updates	Socket.IO events, live timeline, notifications
3	Give officers a secure portal to review, verify and decide	Admin queue, review screen, audit log
4	Assist students with explainable guidance	Smart Assistant, compatibility engine, simulator, chatbot
5	Widen access beyond one college and one language	Global directory, one-stop application, language switcher
6	Secure every request	JWT, bcrypt, role-based middleware
7	Keep every intelligent feature explainable	Reasons and disclaimers shown with every score
8	Verify the main flows automatically	API smoke tests, WebSocket test
9	Work on any device	Responsive React interface

Table 1.1 Objectives of the project

Objectives 1 to 3 form the core admission workflow, while objectives 4 to 9 add guidance, reach and reliability around it.

Together these objectives aim at a platform that is simple for students, efficient for admission officers and transparent for the institution, while every admission decision stays with a human officer. The following chapters describe how each objective was met.

1.4 SCOPE OF THE PROJECT

The project covers the admission life cycle from the first visit to enrolment. Table 1.2 summarises what is included and what is left for future work.

Within scope	Outside the present scope
Registration, login and digital profile	Online payment of fees
Document upload and verification	Government mark-sheet database integration
Application wizard and live tracking	Delivery of applications to other universities' systems
Course, scholarship and analytics management	Native mobile application
Fairness audit and policy simulator	Automatic admission decisions
Global directory, one-stop applications, languages, chatbot	Video interviews

Table 1.2 Scope of the project

Applications made to universities from the global directory are recorded and tracked inside SmartApply. Compatibility scores, simulations and chatbot replies are decision-support aids and never decide admission.

The system is intended for undergraduate and postgraduate admissions in a single institution or a group of institutions, and it can be deployed on any server that runs Node.js. Its modular design allows the administrator to add courses, scholarships and colleges without changes to the code.

The intended users are applicants, admission officers and department staff who review applications. Parents can follow the progress of an application through the student's account.

The project also includes demonstration data, with a sample student and an admission officer account, so that every module can be shown without entering data by hand. This also makes the system suitable for training new admission staff.

1.5 MOTIVATION

The motivation for the project came from the experience of applying to colleges: filling in the same details many times, carrying photocopies of certificates and waiting without information. A first-generation applicant has even less guidance about which course or scholarship suits them.

The technology needed to solve these problems is mature and freely available. Building SmartApply was an opportunity to apply it to a real problem, and to show that intelligent features can be added transparently while a human officer keeps the final authority.

Another motivation was the gap between how students use technology and how admissions still work. Students track parcels, payments and deliveries in real time on their phones, yet the most important application of their academic life is often followed by calling an office. Bringing the same immediacy to admissions seemed both useful and achievable.

Finally, the project was an opportunity to work with the complete MERN stack on a realistic problem, covering database design, secure REST APIs, real-time communication and a responsive interface, and to think carefully about fairness when software assists in decisions that affect people’s futures.

Conversations with friends who had recently gone through admissions confirmed these points. Most had missed at least one update or deadline, and many had applied to courses without knowing whether they were eligible. These experiences decided the features SmartApply puts first: live tracking, early error detection and clear guidance.

CHAPTER 2
RELATED WORK

2.1 LITERATURE REVIEW

The design of SmartApply draws on existing admission systems and on published work about the technologies it uses.

Centralised admission portals

Shared application systems such as the Common Application in the United States and UCAS in the United Kingdom allow a student to enter details once and apply to several institutions. In India, centralised counselling portals such as JoSAA for the IITs and NITs and TNEA for engineering colleges in Tamil Nadu handle registration, choice filling and seat allotment online. These systems prove that a single profile reused across institutions reduces effort. Their limitation, from the point of view of this project, is that status information is usually released in scheduled rounds and the student has to log in and check; they are also specific to one country or one group of institutions.

Web architecture and the MERN stack

Fielding's description of the REST architectural style [1] is the basis of the API design used here: resources are identified by URLs, operated on with standard HTTP methods and exchanged as stateless JSON messages. The MERN stack combines MongoDB [2], Express.js [3], React [4] and Node.js [5], so that the same language is used on the client and the server. MongoDB's document model suits admission data, in which an application naturally embeds an academic snapshot, a list of documents and audit results; Mongoose [6] adds schemas and validation on top of it.

Real-time communication

The WebSocket protocol, standardised as RFC 6455 [7], keeps one two-way connection open between browser and server, which removes the need for

the client to poll for changes. Socket.IO [8] builds on it and adds automatic reconnection and rooms, which let the server address one user or one application. This is the mechanism used for live tracking in SmartApply.

Authentication and password storage

JSON Web Tokens, defined in RFC 7519 [9], carry a signed set of claims so that the server can authenticate each request without keeping session state. For password storage, Provos and Mazières proposed bcrypt [10], an adaptive hashing scheme whose cost can be raised as hardware becomes faster; it remains a standard choice and is used in this project.

Fairness in selection

The four-fifths rule in the Uniform Guidelines on Employee Selection Procedures [11] states that a selection rate for any group which is less than four-fifths of the rate for the group with the highest rate is regarded as evidence of adverse impact. Although written for employment, the same ratio is widely used as a first screening measure in studies of algorithmic fairness, and it is the measure adopted by the Fairness Audit module.

Area	What existing work provides	Gap addressed by SmartApply
Centralised portals	Single form for many institutions; online allotment	Instant status push, guidance modules, multilingual access
College websites	Online form submission	Tracking, audit trail and reuse of profile
Real-time web	WebSocket and Socket.IO rooms	Applied to per-student and per-application updates
Decision support	Rule-based scoring, four-fifths rule	Explainable scores with human-in-the-loop control

Table 2.1 Summary of the literature review

The review therefore shaped three design decisions: a single reusable profile, push-based status updates, and explainable decision support under human control.

2.2 EXISTING SYSTEM

In the existing system followed by many colleges, the student collects or downloads an application form, fills it by hand or in a basic web form, attaches photocopies of certificates and submits it at the office or by post. The office enters the details into a spreadsheet or a standalone package, verifies the certificates manually and informs the selected students by notice board, phone, SMS or e-mail.

Even where colleges have moved to online forms, the change is often limited to collecting data. Verification, status changes and communication remain manual, so the student's experience after submission is the same as with paper. A complete solution must digitise the whole workflow, not only the form.

The disadvantages of the existing system are:

- The student cannot see the current stage of the application.
- Details and documents must be submitted again for each course or college.
- Errors in marks and missing documents are found late and corrected slowly.
- There is no record of who changed a status and why.
- Students receive no personalised guidance on courses or scholarships.
- The office has no analytics on demand, pipeline or consistency of decisions.
- Portals, where they exist, are available in a single language.
- Applying to institutions in other regions or countries means learning a different process for each.
- Enquiries in the peak season overload the admission office's phone lines and front desk.

2.3 PROPOSED SYSTEM

The proposed system, SmartApply, replaces these steps with one web platform. The student keeps one profile and one document locker, applies in a guided wizard and watches the application move through its stages in real time. The admission officer works from a queue, opens an application, sees the assistant's findings beside the documents and records the decision with remarks; the system writes the history, creates the notification and pushes the update to the student.

The advantages of the proposed system over the existing system are compared in Table 2.2.

Feature	Existing system	Proposed system
Application	Paper or basic web form	Guided wizard with saved profile
Status updates	Notice board, phone or e-mail	Instant push through Socket.IO
Error detection	During manual verification	Smart Assistant, before submission
Many institutions	Separate form for each	One-stop application, up to ten
Guidance	None	Scores, simulator and chatbot
Audit trail	Not recorded	Full history with officer and remarks
Language	Single language	Over a hundred, with right-to-left
Security	Shared office records	JWT, bcrypt and role-based access

Table 2.2 Comparison of existing and proposed system

For the admission officer, the main change is that routine work is handled by the system: notifications, history and audit entries are created automatically, so the officer can concentrate on reviewing the application itself. Students, in turn, no longer need to visit or call the office to learn where their application stands.

CHAPTER 3
SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

The requirements were identified from the steps of the manual admission process and from the needs of its two main users, the student and the admission officer. They are grouped into functional requirements, which describe what the system does, and non-functional requirements, which describe how well it does it.

Functional Requirements

Student

- Register, log in and log out; the session is restored from a stored token.
- Create and edit a digital profile with personal details, 10th and 12th marks, entrance examinations, skills and interests.
- Upload, view and replace documents in the document locker.
- Browse courses, view compatibility recommendations and run the what-if simulator.
- Apply to a course through the application wizard and receive assistant warnings before submission.
- Browse the global university directory by region, country and name; shortlist universities and apply to all of them in one step.
- Track each application on a live timeline and receive notifications.
- View matched scholarships, change the display language and ask the counsellor chatbot.

Administrator (admission officer)

- View the dashboard with application counts, pipeline distribution and demand per course.
- Filter and search the application queue; open an application for review.

- Verify or flag documents; change application status with remarks; schedule an interview.
- Create, update and archive courses and scholarships.
- Run the fairness audit and the policy simulator; view analytics.

System

- Authenticate every protected request and allow access according to role.
- Record status history and audit logs; create and deliver notifications in real time.
- Synchronise the university directory from the open data source periodically, with a saved copy as fallback.
- Generate a unique application number for every submitted application.
- Notify the student of every status change, document verification and interview schedule.

Non-Functional Requirements

Requirement	Description
Security	Passwords are hashed with bcrypt; tokens expire after seven days; admin routes reject non-admin users; the password field is never returned by queries.
Performance	Status updates reach the connected client within a second on a local network; directory search is served from memory with pagination.
Usability	Responsive layout for desktop and mobile; one-click demo accounts; plain status descriptions; right-to-left layout for languages that need it.
Reliability	If no MongoDB server is available, an embedded in-memory instance is started and seeded; the directory falls back to a saved or built-in list.
Maintainability	Code is separated into models, controllers, routes, middleware and services on the server and pages, components, layouts and contexts on the client.
Transparency	Every score and recommendation lists its reasons and carries a notice that it does not guarantee admission.

Table 3.1 Non-functional requirements

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements

Component	Specification
Operating system	Windows 10 or later, macOS, or Linux
Runtime	Node.js version 18 or later, npm version 9 or later
Front end	React 18, React Router 6, Vite 6, Tailwind CSS 3, Axios, Recharts, Lucide icons, Socket.IO client
Back end	Express.js 4, Mongoose 8, Socket.IO 4, jsonwebtoken, bcryptjs, Multer, Morgan, dotenv, CORS
Database	MongoDB (local or Atlas); mongodb-memory-server as embedded fallback
Tools	Visual Studio Code, Git, a modern web browser (Chrome, Edge, Firefox or Safari)
Deployment	Any Node.js host; a render.yaml file is included for Render

Table 3.2 Software requirements

Hardware Requirements

Component	Minimum specification
Processor	Intel Core i3 / AMD Ryzen 3 or equivalent, 2.0 GHz
Memory	4 GB RAM (8 GB recommended for development)
Storage	2 GB free disk space for the project, dependencies and uploads
Network	Internet connection for package installation, directory synchronisation and translation
Client device	Any desktop, laptop, tablet or smartphone with a modern browser

Table 3.3 Hardware requirements

All the software listed above is free and open source, so the project can be developed and run without licence costs. For a production deployment, a managed MongoDB service such as MongoDB Atlas and a Node.js hosting platform are recommended.

CHAPTER 4
SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

SmartApply follows a three-tier architecture. The client tier is a single-page React application. The server tier is a Node.js process in which Express handles REST requests and Socket.IO handles real-time events on the same HTTP server. The data tier is MongoDB, accessed through Mongoose models, together with a folder for uploaded documents.

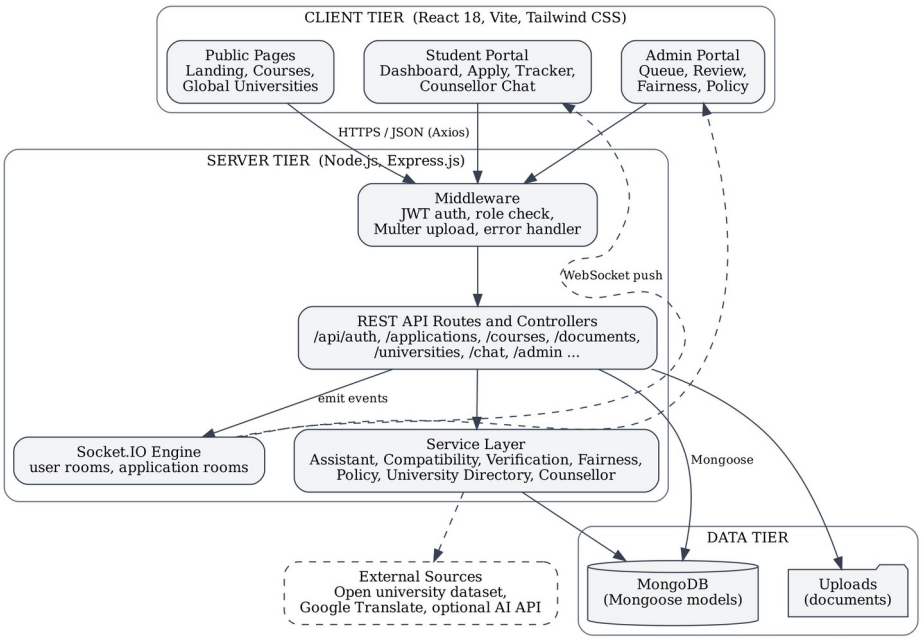


Figure 4.1 System architecture of SmartApply

A request from the browser carries a JSON Web Token in the Authorization header. The authentication middleware verifies the token and loads the user, and the authorisation middleware checks the role. The controller then reads or writes MongoDB, calls a service where rule-based logic is needed, and returns JSON. When the change concerns an application, the controller also emits a Socket.IO event to the room of the affected student, so that the student's open pages update immediately.

4.2 MODULE DESCRIPTION

1. Authentication and Authorisation Module

Handles registration, login and session restoration. Passwords are hashed in a Mongoose pre-save hook. Login returns a signed token that the client stores and attaches to every request. Two middleware functions, protect and authorize, guard the routes by identity and by role (student or admin).

2. Student Profile and Document Locker Module

Stores personal information, 10th and 12th details with subject-wise marks, entrance examination scores, skills, interests and career goals, and computes a completion percentage. Documents are uploaded with Multer, classified by type and given a verification status. The rule-based verification engine checks type, size and consistency and marks a document as verified or flagged for the officer.

3. Application and Real-Time Tracking Module

The application wizard collects the course, an academic snapshot and the documents. On submission the server generates an application number, stores the assistant audit, writes the first history entry and notifies the student. The application then moves through the stages Draft, Submitted, Document Verification, Correction Required, Academic Review, Interview, Waitlisted, Approved, Rejected and Enrolled.

4. Smart Application Assistant Module

Compares the percentage entered by the student with the percentage computed from subject marks, checks stream and subject requirements of the course and lists missing mandatory documents. Findings are graded as information, warning or critical and are shown to both student and officer. The assistant never rejects an application.

5. Course Compatibility, Simulator and Scholarship Module

The compatibility engine scores each course out of 100 from academic cut-off, required subjects, entrance score, skills, interests and career goals, and labels it Recommended (70 and above), Possible (50 to 69) or Not Recommended. The what-if simulator recalculates readiness as marks change. The scholarship matcher checks percentage, family income, category, stream and gender rules and reports the reasons and the gaps.

6. Administrator Module

Provides the dashboard, the filterable application queue, the review screen, course and scholarship management and analytics. The fairness audit computes selection rates by stream and score group and their four-fifths ratios. The policy simulator estimates the effect of changing seats, cut-offs or deadlines on demand and waitlists.

7. Global University Directory and One-Stop Application Module

The server downloads an open list of world universities, groups it into seven regions and refreshes it every twelve hours, keeping a saved copy for offline use. The student can shortlist up to ten universities and apply to all of them with one request; each becomes a normal application that is tracked like any other.

8. Multilingual Interface Module

A language switcher in the navigation bar offers more than a hundred languages through automatic page translation and sets the page direction to right-to-left for languages such as Arabic and Urdu.

9. Counsellor Chatbot Module

A chat window on every student page answers questions about application status, suitable courses, scholarships, missing documents, deadlines, interviews and universities in a given country. The server builds each answer from the student’s own records and stores the conversation in MongoDB.

Table 4.1 summarises the modules with their main server files and the users they serve.

Module	Main server files	Users
Authentication	authController, middleware/auth	All
Profile and documents	profileController, documentController	Student
Application tracking	applicationController, config/socket	Student, officer
Smart Assistant	assistantEngine, verificationEngine	Student, officer
Guidance	compatibilityEngine, simulator, scholarship	Student
Administrator	adminController, fairnessEngine, policyEngine	Officer
Global directory	universityDirectory, universityController	Visitor, student
Multilingual interface	LanguageSwitcher (client)	All
Counsellor chatbot	counsellorEngine, chatController	Student

Table 4.1 Summary of modules

The modules communicate only through the REST API and Socket.IO events, so each can be changed or extended without affecting the others. This separation also made it possible to add the global directory, the multilingual interface and the chatbot late in the project without modifying the existing modules.

During development, the server logged every request through Morgan, which made it easy to trace a problem from the user interface through the API to the module responsible.

4.4 CLASS DIAGRAM

The class diagram shows the main Mongoose models and their relationships. A User has one StudentProfile and many Applications, Documents, Notifications and ChatMessages. A College offers many Courses, and each Application refers to one Course and one College and owns its status history.

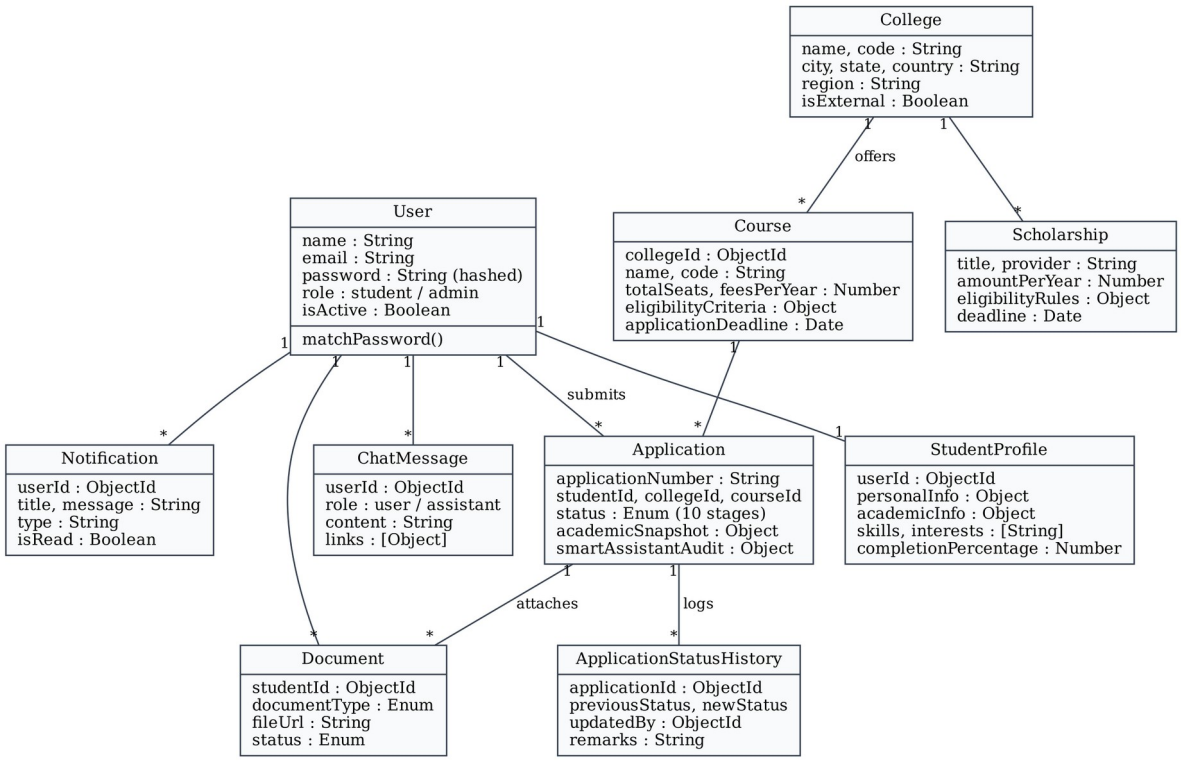


Figure 4.2 Class diagram of the data model

The ApplicationStatusHistory model is the basis of the live timeline: each record stores the previous and new status, the officer who made the change and the remarks. The Notification and ChatMessage models belong to the user and are created by the server whenever an application changes or the student asks the counsellor a question.

All relationships are stored as ObjectId references and resolved with Mongoose populate only when a page needs the related data.

4.5 SEQUENCE DIAGRAM

The sequence diagram shows the two central interactions: a student submitting an application, and an admission officer changing its status, after which the update is delivered to the student in real time.

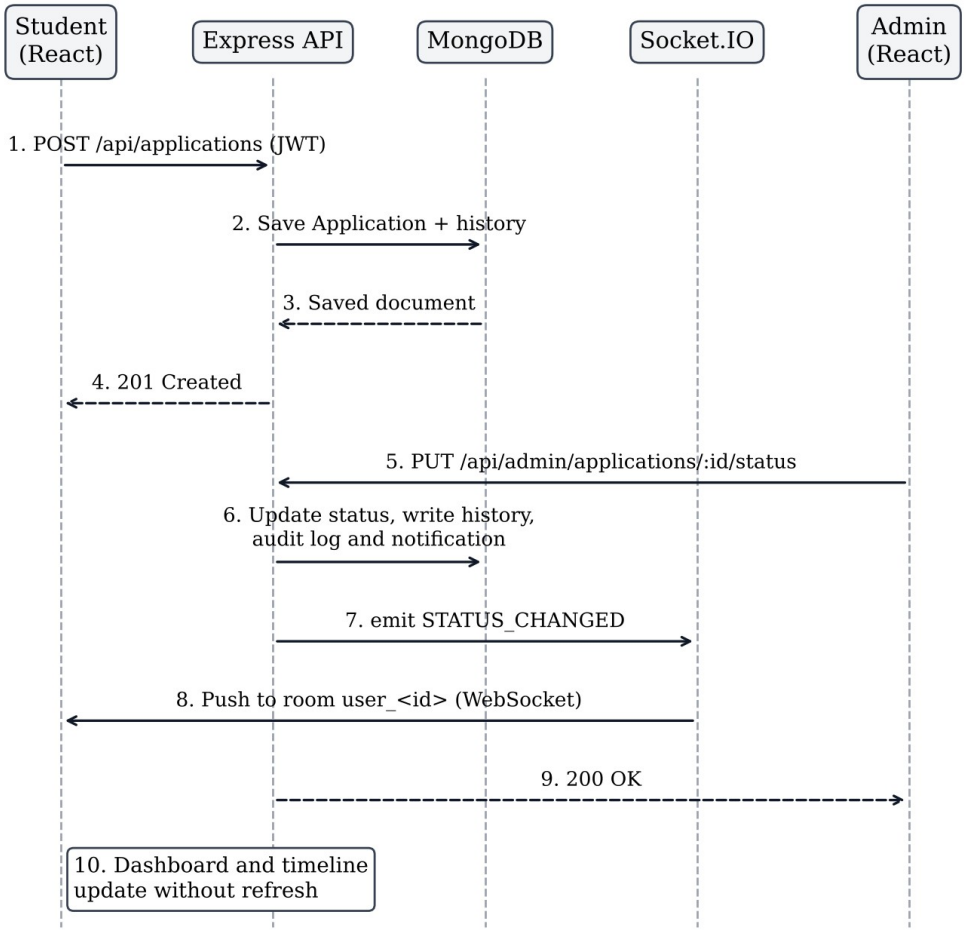


Figure 4.3 Sequence diagram for application submission and real-time status update

Steps 1 to 4 form the ordinary request–response cycle of the REST API. Steps 5 to 10 show the real-time path: the officer’s change is saved first and then pushed to the student’s room, so the dashboard updates without any request from the browser.

CHAPTER 5
IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

The project is organised as a monorepo with two folders, server and client, and a root package.json whose scripts install, build and start both parts. Development was carried out in stages: data models and seed data first, then authentication, the REST API, the React pages, the real-time layer and finally the decision-support and extension modules.

Back end. The server follows the model–route–controller pattern. Twelve Mongoose models define the collections. Each feature has a route file that maps URLs to controller functions and applies the protect and authorize middleware. Logic that is not simple data access is kept in the services folder: assistantEngine, compatibilityEngine, verificationEngine, fairnessEngine, policyEngine, universityDirectory and counsellorEngine. A central error handler converts exceptions into JSON responses. On start-up the server connects to MongoDB, seeds demonstration data if the database is empty, starts the directory synchronisation and listens on port 5001.

Front end. The React application is built with Vite and styled with Tailwind CSS. React Router defines three groups of routes under PublicLayout, StudentLayout and AdminLayout; the two portal layouts redirect to the login page when the user is absent or has the wrong role. Three context providers hold shared state: AuthContext (user, token and login functions), SocketContext (the Socket.IO connection and room joining) and NotificationContext (the notification list and unread count). A single Axios instance adds the token to every request and handles expired sessions.

Real-time layer. When a user logs in, the client connects to Socket.IO and joins the room user_<id>; the tracker page also joins app_<id>. Controllers call

emitToUser and emitToApplication after writing to the database, and the client's listeners refresh the dashboard, timeline and notification drawer.

Testing. A smoke-test script calls the main endpoints as student and as administrator, and a socket test verifies that a status change made through the admin API arrives as a STATUS_CHANGED event on the student's connection. The new modules were tested through their endpoints and in the browser on desktop and mobile widths.

API group	Base path	Purpose
Authentication	/api/auth	Register, login, current user
Profile	/api/profile	Read and update student profile
Courses	/api/courses	List, view, create, update, archive
Applications	/api/applications	Assistant check, create, list, view
Documents	/api/documents	Upload, list, verify
Recommendations	/api/recommendations	Course compatibility scores
Simulator	/api/simulator	What-if readiness
Scholarships	/api/scholarships	List, match, create
Notifications	/api/notifications	List and mark as read
Universities	/api/universities	Directory search, meta, one-stop apply
Chat	/api/chat	Counsellor history and messages
Admin	/api/admin	Stats, queue, status, fairness, policy

Table 5.1 REST API groups

The root package.json defines the scripts used to run the project: npm install installs both parts and builds the client, npm start runs the unified server on port 5001, and npm run server and npm run client start the back end and the Vite development server separately during development.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

Password hashing in the User model

The password is never stored as plain text. A pre-save hook runs before a user document is written; if the password field has changed, it generates a salt with ten rounds and replaces the password with its bcrypt hash. The schema also marks the field select: false, so ordinary queries do not return it. The matchPassword method compares a login attempt with the stored hash.

```
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};
```

Route protection with protect and authorize

protect reads the Bearer token, verifies its signature and expiry, loads the user and attaches it to the request. authorize is a function that returns a middleware for the roles passed to it, which makes route definitions short and readable: router.use(protect, authorize('admin')) secures a whole admin router in one line.

```
const authorize = (...roles) => {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({ success: false,
        message: 'User role is not authorized to access this resource.' });
    }
    next();
  };
};
```

Status transition with audit trail and real-time event

updateApplicationStatus is the function behind live tracking. It saves the new status, writes an ApplicationStatusHistory entry and an AuditLog entry, creates a Notification with a friendly message, and finally emits two events to the

student’s room. Because the database is written before the events are sent, a client that misses the event still sees the correct state on its next request.

```
const previousStatus = application.status;
application.status = status;
if (remarks) application.adminRemarks = remarks;
await application.save();

await ApplicationStatusHistory.create({
  applicationId: application._id, previousStatus, newStatus: status,
  updatedBy: req.user.id, remarks
});

emitToUser(application.studentId._id.toString(), 'NOTIFICATION_RECEIVED', notif);
emitToUser(application.studentId._id.toString(), 'STATUS_CHANGED', {
  applicationId: application._id, previousStatus, newStatus: status,
  remarks, timestamp: new Date()
});
```

Socket.IO rooms

Each connected client asks to join a private room. emitToUser then addresses that room only, so one student never receives another student’s updates.

```
io.on('connection', (socket) => {
  socket.on('join_user', (userId) => {
    if (userId) socket.join(`user_${userId}`);
  });
  socket.on('join_application', (applicationId) => {
    if (applicationId) socket.join(`app_${applicationId}`);
  });
});

const emitToUser = (userId, event, data) => {
  if (io && userId) io.to(`user_${userId}`).emit(event, data);
};
```

One-stop application to several universities

applyToUniversities accepts a list of university keys and a program name. For each key it looks the university up in the directory (so that only real entries are accepted), creates the College and Course records if they do not exist, skips universities to which the student already has an active application, and creates the Application with the same academic snapshot and documents. The response lists the result for every university.

```
for (const rawKey of universityKeys) {
  const uni = await directory.findByKey(rawKey);
  if (!uni) { results.push({ key: rawKey, success: false,
```

```
    message: 'University not found in the directory.' }); continue; }

  const college = await College.findOneAndUpdate(
    { code: `GU-${hash}` },
    { $setOnInsert: { name: uni.name, country: uni.country,
        region: uni.region, isExternal: true /* ... */ } },
    { new: true, upsert: true });

  const application = await Application.create({
    applicationNumber: await uniqueAppNumber(),
    studentId: req.user.id, collegeId: college._id, courseId: course._id,
    status: 'SUBMITTED', academicSnapshot, submittedAt: new Date()
  });
  results.push({ key: rawKey, success: true,
    applicationNumber: application.applicationNumber });
}
```

Counsellor chatbot reply

generateReply first builds a context object containing the student's profile, applications, documents, course matches and eligible scholarships. The rule-based function then selects an answer by matching keywords in the question. If an AI key is configured the model phrases the reply, with the rule-based answer as fallback.

```
const generateReply = async (user, message, history = []) => {
  const ctx = await buildContext(user);
  const base = await rulesReply(ctx, message);
  const reply = { text: base.text, links: base.links || [],
                  suggestions: base.suggestions, source: 'rules' };
  if (isAiEnabled()) {
    try {
      reply.text = await aiReply(ctx, message, history);
      reply.source = 'ai';
    } catch (err) { /* keep the built-in answer */ }
  }
  return reply;
};
```

Together these functions cover the three central concerns of the system: secure access, reliable real-time tracking, and assistance that always remains under human control.

CHAPTER 6
CONCLUSION & FUTURE WORK

6.1 CONCLUSION

SmartApply shows that the college admission process can be moved fully online with the MERN stack and that doing so removes the main difficulties of the manual process. A student maintains one profile and one set of documents, applies in a few steps and sees every change of status at the moment it happens. An admission officer works from an organised queue, and each decision is recorded with its remarks, time and author.

The project met its objectives. Table 6.1 lists the main tests that were run on the completed system and their results.

Test	Expected result	Result
Student and admin login	Token issued for each role	Pass
Request without token	Rejected with status 401	Pass
Course catalogue	All active courses listed	Pass
Course recommendations	Ranked matches with reasons	Pass
What-if simulator	Readiness recalculated	Pass
Scholarship matching	Eligible scholarships listed	Pass
Fairness audit, policy simulator	Reports generated	Pass
Real-time status update	Student receives event instantly	Pass
One-stop application	Applications created, duplicate refused	Pass
Counsellor chatbot	Correct answer for each question type	Pass

Table 6.1 Test results

Equally important, the project kept every intelligent feature transparent. Each score, warning and chatbot answer states its reasons, and no feature can approve or reject an application. This keeps the admission officer accountable for every decision while still reducing the routine work involved in reaching it.

6.2 FUTURE ENHANCEMENTS

The following enhancements are planned.

- **Online fee payment** with a payment gateway and automatic receipts, and a downloadable offer letter in PDF.
- **Mark-sheet reading by OCR**, so that marks are filled in automatically from an uploaded image and compared with the entered values.
- **E-mail, SMS and WhatsApp notifications** in addition to in-app alerts, and a deadline calendar with reminders.
- **Cost and visa planner** for universities abroad, with tuition and living cost estimates.
- **Verified digital certificates** carrying a QR code that any institution can scan to confirm authenticity.
- **Built-in video interviews** linked to the Interview stage of an application.
- **Hand-checked translations** for major Indian languages in place of automatic translation, and voice input for form filling.
- **Machine-learning models** trained on past admission data to forecast demand and acceptance, kept under the same human-in-the-loop policy.
- **A mobile application** built with React Native that reuses the existing API.
- **Applicant insights** showing, in anonymised form, how a profile compares with previously admitted cohorts.
- **Accessibility improvements** such as screen-reader labels, keyboard navigation and a high-contrast theme.
- **Progressive web app** support so that students with weak connectivity can still view their status.
- **Direct integration** with the admission systems of partner universities so that one-stop applications are delivered to them electronically.

APPENDIX

SOURCE CODE:

1. Project Structure

Project Folder Structure

```
smartapply-main/
|-- server/
|   |-- config/           db.js, jwt.js, socket.js
|   |-- controllers/
|   |   |-- authController.js          applicationController.js
|   |   |-- courseController.js        documentController.js
|   |   |-- profileController.js       recommendationController.js
|   |   |-- simulatorController.js     scholarshipController.js
|   |   |-- notificationController.js  adminController.js
|   |   `-- universityController.js    chatController.js
|   |-- middleware/        auth.js, upload.js, errorHandler.js
|   |-- models/
|   |   |-- User.js, StudentProfile.js, College.js, Course.js
|   |   |-- Application.js, ApplicationStatusHistory.js, Document.js
|   |   `-- Scholarship.js, Notification.js, AuditLog.js, Simulation.js, ChatMessage.js
|   |-- routes/           (one route file per controller)
|   |-- services/
|   |   |-- assistantEngine.js, compatibilityEngine.js, verificationEngine.js
|   |   `-- fairnessEngine.js, policyEngine.js, universityDirectory.js, counsellorEngine.js
|   |-- seeds/            seed.js, seedHelper.js
|   |-- tests/           smokeTest.js, socketTest.js
|   |-- uploads/
|   |-- package.json
|   `-- server.js
|-- client/
|   |-- src/
|   |   |-- components/    Navbar, Timeline, StatusBadge, Modal, LanguageSwitcher, CounsellorChat ...
|   |   |-- context/       AuthContext, SocketContext, NotificationContext
|   |   |-- layouts/       PublicLayout, StudentLayout, AdminLayout
|   |   |-- pages/
|   |   |   |-- public/    Landing, Courses, Login, Register, GlobalUniversities
|   |   |   |-- student/   Dashboard, Profile, ApplicationWizard, ApplicationDetails, DocumentLocker ...
|   |   |   `-- admin/     Dashboard, Applications, Review, Courses, FairnessAudit, PolicySimulator ...
|   |   |-- services/      api.js (Axios instance with interceptors)
|   |   |-- App.jsx
|   |   `-- main.jsx
|   |-- index.html
|   |-- vite.config.js
|   `-- package.json
|-- .env.example
|-- render.yaml
|-- package.json
`-- README.md
```

2. Backend Package Installation

```
# create the server project and install dependencies
```

```
mkdir server && cd server
npm init -y
npm install express mongoose dotenv cors morgan
npm install jsonwebtoken bcryptjs multer socket.io
npm install mongodb-memory-server

# create the React client with Vite and install dependencies
npm create vite@latest client -- --template react
cd client
npm install axios react-router-dom socket.io-client recharts lucide-react
npm install tailwindcss postcss autoprefixer

# run the complete application from the project root
npm install        # installs server and client, builds the client
npm start          # http://localhost:5001
```

3. Database Connection

server/config/db.js

```
const mongoose = require('mongoose');
const { MongoMemoryServer } = require('mongodb-memory-server');

let mongodInstance = null;

const connectDB = async () => {
  const uri = process.env.MONGODB_URI;

  if (uri && !uri.includes('memory')) {
    try {
      await mongoose.connect(uri, { serverSelectionTimeoutMS: 2500 });
      console.log('MongoDB Connected successfully to specified URI.');
      return;
    } catch (err) {
      console.warn(`Could not connect to ${uri}. Falling back to embedded server...`);
    }
  }

  // Fallback to an embedded in-memory instance
  try {
    mongodInstance = await MongoMemoryServer.create();
    await mongoose.connect(mongodInstance.getUri());
    console.log('MongoDB Connected successfully to embedded database.');
  } catch (error) {
    console.error('Fatal: Failed to connect to MongoDB', error);
    process.exit(1);
  }
};

module.exports = { connectDB };
```

4. Environment Variables

.env

```
PORT=5001
NODE_ENV=development
MONGODB_URI=mongodb://localhost:27017/smartapply
```

```
JWT_SECRET=<a long random secret string>
CLIENT_URL=http://localhost:5173

# Optional: AI-written replies for the counsellor chatbot
# ANTHROPIC_API_KEY=
```

5. User Model

server/models/User.js

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name:     { type: String, required: [true, 'Name is required'], trim: true },
  email:    { type: String, required: [true, 'Email is required'],
              unique: true, lowercase: true, trim: true, index: true },
  password: { type: String, required: [true, 'Password is required'],
              minlength: 6, select: false },
  role:     { type: String, enum: ['student', 'admin'], default: 'student', index: true },
  phone:    { type: String, trim: true },
  avatar:   { type: String, default: '' },
  collegeId:{ type: mongoose.Schema.Types.ObjectId, ref: 'College' },
  isActive: { type: Boolean, default: true }
}, { timestamps: true });

// Pre-save password hash
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Compare password
userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

6. Password Hashing and Registration

server/controllers/authController.js (the password is hashed by the pre-save hook shown above)

```
exports.register = async (req, res, next) => {
  try {
    const { name, email, password, role, phone, collegeId } = req.body;

    const existingUser = await User.findOne({ email: email.toLowerCase() });
    if (existingUser) {
      return res.status(400).json({
        success: false,
        message: 'A user with this email address is already registered.'
      });
    }
```

```
    const assignedRole = role === 'admin' ? 'admin' : 'student';
    const user = await User.create({
      name, email: email.toLowerCase(), password, role: assignedRole, phone,
      collegeId: assignedRole === 'admin' ? collegeId : undefined
    });

    // Create the initial StudentProfile for a new student
    if (assignedRole === 'student') {
      await StudentProfile.create({ userId: user._id, completionPercentage: 35 });
    }

    const token = signToken({ id: user._id, role: user.role });
    res.status(201).json({
      success: true, token,
      user: { id: user._id, name: user.name, email: user.email, role: user.role }
    });
  } catch (error) {
    next(error);
  }
};
```

7. Login and JWT Generation

server/config/jwt.js and server/controllers/authController.js

```
const jwt = require('jsonwebtoken');
const JWT_SECRET = process.env.JWT_SECRET;
const JWT_EXPIRES_IN = '7d';

const signToken = (payload) => jwt.sign(payload, JWT_SECRET, { expiresIn:
JWT_EXPIRES_IN });

const verifyToken = (token) => {
  try { return jwt.verify(token, JWT_SECRET); } catch (err) { return null; }
};

exports.login = async (req, res, next) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return res.status(400).json({ success: false,
        message: 'Please provide an email and password.' });
    }

    const user = await User.findOne({ email: email.toLowerCase() }).select('+password');
    if (!user || !(await user.matchPassword(password))) {
      return res.status(401).json({ success: false,
        message: 'Invalid credentials. Please verify your email and password.' });
    }

    const token = signToken({ id: user._id, role: user.role });
    res.status(200).json({
      success: true, token,
      user: { id: user._id, name: user.name, email: user.email, role: user.role }
    });
  } catch (error) {
    next(error);
  }
```

```
};
```

8. Authentication Middleware

server/middleware/auth.js

```
const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    token = req.headers.authorization.split(' ')[1];
  }
  if (!token) {
    return res.status(401).json({ success: false,
      message: 'Not authorized to access this route. Please log in.' });
  }

  const decoded = verifyToken(token);
  if (!decoded) {
    return res.status(401).json({ success: false,
      message: 'Invalid or expired session token.' });
  }

  const user = await User.findById(decoded.id);
  if (!user || !user.isActive) {
    return res.status(401).json({ success: false,
      message: 'The user belonging to this token no longer exists or is inactive.' });
  }
  req.user = user;
  next();
};

const authorize = (...roles) => (req, res, next) => {
  if (!req.user || !roles.includes(req.user.role)) {
    return res.status(403).json({ success: false,
      message: 'User role is not authorized to access this resource.' });
  }
  next();
};

module.exports = { protect, authorize };
```

9. Create API

server/controllers/courseController.js – create a course (admin)

```
exports.createCourse = async (req, res, next) => {
  try {
    const course = await Course.create(req.body);
    res.status(201).json({ success: true, course });
  } catch (error) {
    next(error);
  }
};
```

10. Read API

List courses with optional filters and search

```
exports.getCourses = async (req, res, next) => {
```

```
  try {
    const { department, degreeLevel, search, collegeId } = req.query;
    const query = { isActive: true, isExternal: { $ne: true } };

    if (department) query.department = department;
    if (degreeLevel) query.degreeLevel = degreeLevel;
    if (collegeId) query.collegeId = collegeId;
    if (search) {
      query.$or = [
        { name: { $regex: search, $options: 'i' } },
        { code: { $regex: search, $options: 'i' } },
        { department: { $regex: search, $options: 'i' } }
      ];
    }

    const courses = await Course.find(query)
      .populate('collegeId', 'name code city state logo accreditation');
    res.status(200).json({ success: true, count: courses.length, courses });
  } catch (error) {
    next(error);
  }
};
```

11. Update

```
exports.updateCourse = async (req, res, next) => {
  try {
    const course = await Course.findByIdAndUpdate(
      req.params.id, req.body, { new: true, runValidators: true });
    if (!course) {
      return res.status(404).json({ success: false, message: 'Course not found' });
    }
    res.status(200).json({ success: true, course });
  } catch (error) {
    next(error);
  }
};
```

12. Delete

Courses are archived (soft delete) so that existing applications keep their reference

```
exports.deleteCourse = async (req, res, next) => {
  try {
    const course = await Course.findByIdAndUpdate(
      req.params.id, { isActive: false }, { new: true });
    if (!course) {
      return res.status(404).json({ success: false, message: 'Course not found' });
    }
    res.status(200).json({ success: true, message: 'Course archived successfully' });
  } catch (error) {
    next(error);
  }
};
```

13. Express Routes

server/routes/courseRoutes.js and server/routes/adminRoutes.js

```
const express = require('express');
const router = express.Router();
const { getCourses, getCourseById, createCourse, updateCourse, deleteCourse }
  = require('../controllers/courseController');
const { protect, authorize } = require('../middleware/auth');

router.get('/', getCourses);
router.get('/:id', getCourseById);
router.post('/', protect, authorize('admin'), createCourse);
router.put('/:id', protect, authorize('admin'), updateCourse);
router.delete('/:id', protect, authorize('admin'), deleteCourse);

module.exports = router;

// adminRoutes.js – every route below requires an admin token
router.use(protect, authorize('admin'));
router.get('/stats', getDashboardStats);
router.get('/applications', getApplications);
router.put('/applications/:id/status', updateApplicationStatus);
router.get('/fairness', getFairnessAudit);
router.post('/policy-simulate', runPolicySimulation);
router.get('/analytics', getAnalytics);
```

14. Main Server File

server/server.js

```
const express = require('express');
const http = require('http');
const path = require('path');
const fs = require('fs');
const cors = require('cors');
const morgan = require('morgan');
require('dotenv').config();

const { connectDB } = require('./config/db');
const { initSocket } = require('./config/socket');
const { seedInitialDataIfEmpty } = require('./seeds/seedHelper');
const errorHandler = require('./middleware/errorHandler');

const app = express();
const server = http.createServer(app);
initSocket(server);                    // Socket.IO on the same HTTP server

app.use(cors({ origin: '*', credentials: true }));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(morgan('dev'));
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/profile', require('./routes/profileRoutes'));
app.use('/api/courses', require('./routes/courseRoutes'));
app.use('/api/applications', require('./routes/applicationRoutes'));
app.use('/api/documents', require('./routes/documentRoutes'));
```

```
app.use('/api/recommendations', require('./routes/recommendationRoutes'));
app.use('/api/simulator', require('./routes/simulatorRoutes'));
app.use('/api/scholarships', require('./routes/scholarshipRoutes'));
app.use('/api/notifications', require('./routes/notificationRoutes'));
app.use('/api/admin', require('./routes/adminRoutes'));
app.use('/api/universities', require('./routes/universityRoutes'));
app.use('/api/chat', require('./routes/chatRoutes'));

// Serve the compiled React application
const clientDist = path.join(__dirname, '../client/dist');
if (fs.existsSync(clientDist)) {
  app.use(express.static(clientDist));
  app.get('*', (req, res, next) => {
    if (req.path.startsWith('/api') || req.path.startsWith('/uploads')) return next();
    res.sendFile(path.join(clientDist, 'index.html'));
  });
}

app.use(errorHandler);

const PORT = process.env.PORT || 5001;
const startServer = async () => {
  await connectDB();
  await seedInitialDataIfEmpty();
  server.listen(PORT, () => console.log(`SmartApply is live on port ${PORT}`));
};
startServer();
```

15. React Login Component

client/src/pages/public/LoginPage.jsx and client/src/context/AuthContext.jsx (styling classes omitted)

```
// AuthContext.jsx
const login = async (email, password) => {
  const res = await api.post('/auth/login', { email, password });
  if (res.data.success) {
    localStorage.setItem('smartapply_token', res.data.token);
    localStorage.setItem('smartapply_user', JSON.stringify(res.data.user));
    setUser(res.data.user);
    await fetchCurrentUser();
    return res.data;
  }
};

// LoginPage.jsx
export default function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [submitting, setSubmitting] = useState(false);
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setSubmitting(true);
```

```
    try {
      const res = await login(email, password);
      navigate(res.user.role === 'admin' ? '/admin/dashboard' : '/student/dashboard');
    } catch (err) {
      setError(err.response?.data?.message || 'Invalid email or password.');
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      {error && <div>{error}</div>}
      <input type="email" required value={email}
            onChange={(e) => setEmail(e.target.value)} />
      <input type="password" required value={password}
            onChange={(e) => setPassword(e.target.value)} />
      <button type="submit" disabled={submitting}>
        {submitting ? 'Authenticating...' : 'Sign In'}
      </button>
    </form>
  );
}
```

16. Admin Dashboard Logic

client/src/pages/admin/AdminDashboard.jsx – fetching statistics and recent applications

```
export default function AdminDashboard() {
  const [stats, setStats] = useState(null);
  const [appsPerCourse, setAppsPerCourse] = useState([]);
  const [statusBreakdown, setStatusBreakdown] = useState([]);
  const [recentApplications, setRecentApplications] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => { fetchDashboardStats(); }, []);

  const fetchDashboardStats = async () => {
    setLoading(true);
    try {
      const [statsRes, appsRes] = await Promise.all([
        api.get('/admin/stats'),
        api.get('/admin/applications')
      ]);
      if (statsRes.data.success) {
        setStats(statsRes.data.stats);
        setAppsPerCourse(statsRes.data.appsPerCourse || []);
        setStatusBreakdown(
          statsRes.data.statusBreakdown?.filter(s => s.count > 0) || []);
      }
      if (appsRes.data.success) {
        setRecentApplications(appsRes.data.applications.slice(0, 5));
      }
    } catch (err) {
      console.error('Failed to load admin statistics:', err);
    } finally {
      setLoading(false);
    }
```

```
  };
  // ... charts (Recharts) and the recent applications table are rendered here
}
```

17. REST API Request and Response Examples

Request – login

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "student@smartapply.edu",
  "password": "Student@123"
}
```

Response – 200 OK

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "6acbc1381fbf...",
    "name": "Aarav Patel",
    "email": "student@smartapply.edu",
    "role": "student"
  }
}
```

Request – one-stop application to two universities

```
POST /api/universities/apply
Authorization: Bearer <token>
Content-Type: application/json

{
  "universityKeys": ["oxford brookes university|gb", "university of oxford|gb"],
  "program": "Computer Science",
  "degreeLevel": "Undergraduate"
}
```

Response – 201 Created

```
{
  "success": true,
  "submitted": 2,
  "message": "Submitted 2 applications with your single profile.",
  "results": [
    { "key": "oxford brookes university|gb", "success": true,
      "university": "Oxford Brookes University", "country": "United Kingdom",
      "applicationNumber": "SA-2026-29912" },
    { "key": "university of oxford|gb", "success": true,
      "university": "University of Oxford", "country": "United Kingdom",
      "applicationNumber": "SA-2026-10198" }
  ]
}
```

Request – status change by the admission officer

```
PUT /api/admin/applications/<applicationId>/status
Authorization: Bearer <admin token>
Content-Type: application/json

{
  "status": "ACADEMIC_REVIEW",
  "remarks": "Candidate passed document verification."
}
```

Response – 200 OK (a STATUS_CHANGED event is also pushed to the student)

```
{
  "success": true,
  "message": "Status updated to ACADEMIC_REVIEW.",
  "application": { "applicationNumber": "SA-2026-88102", "status": "ACADEMIC_REVIEW" }
}
```

Request – counsellor chatbot message

```
POST /api/chat/message
Authorization: Bearer <token>
Content-Type: application/json

{ "message": "Which scholarships can I get?" }
```

Response – 201 Created

```
{
  "success": true,
  "reply": {
    "role": "assistant",
    "content": "You meet the eligibility rules for 2 scholarships: ...",
    "links": [{ "label": "Open scholarship matcher", "to": "/student/scholarships" }],
    "suggestions": ["What documents am I missing?", "What is my application status?"],
    "source": "rules"
  }
}
```

SCREENSHOTS OF THE PROJECT

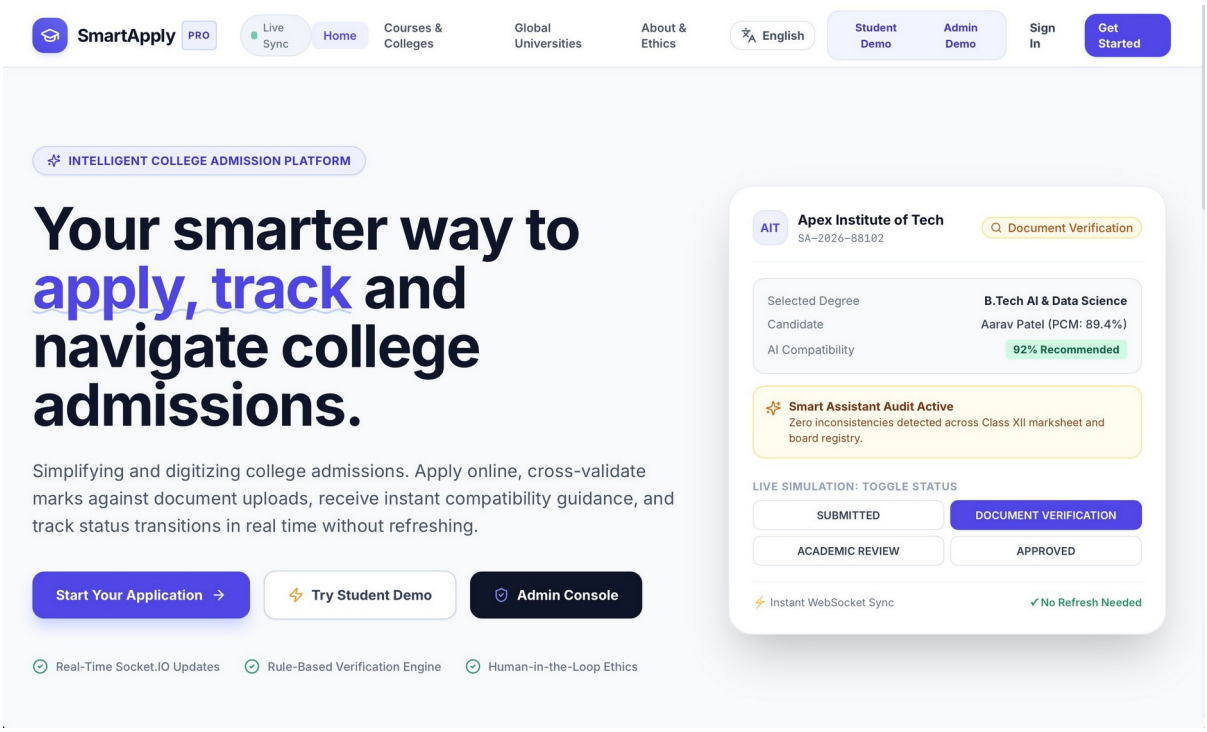


Figure A.1 Landing page

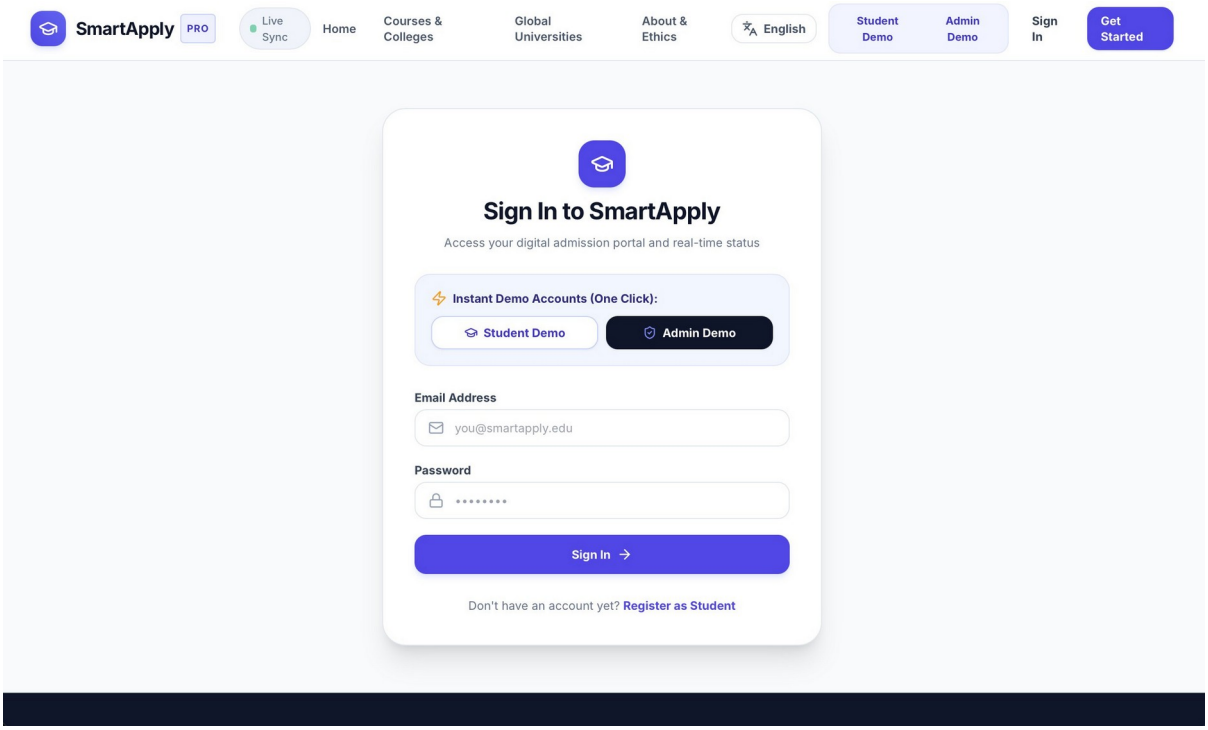


Figure A.2 Login page with demo accounts

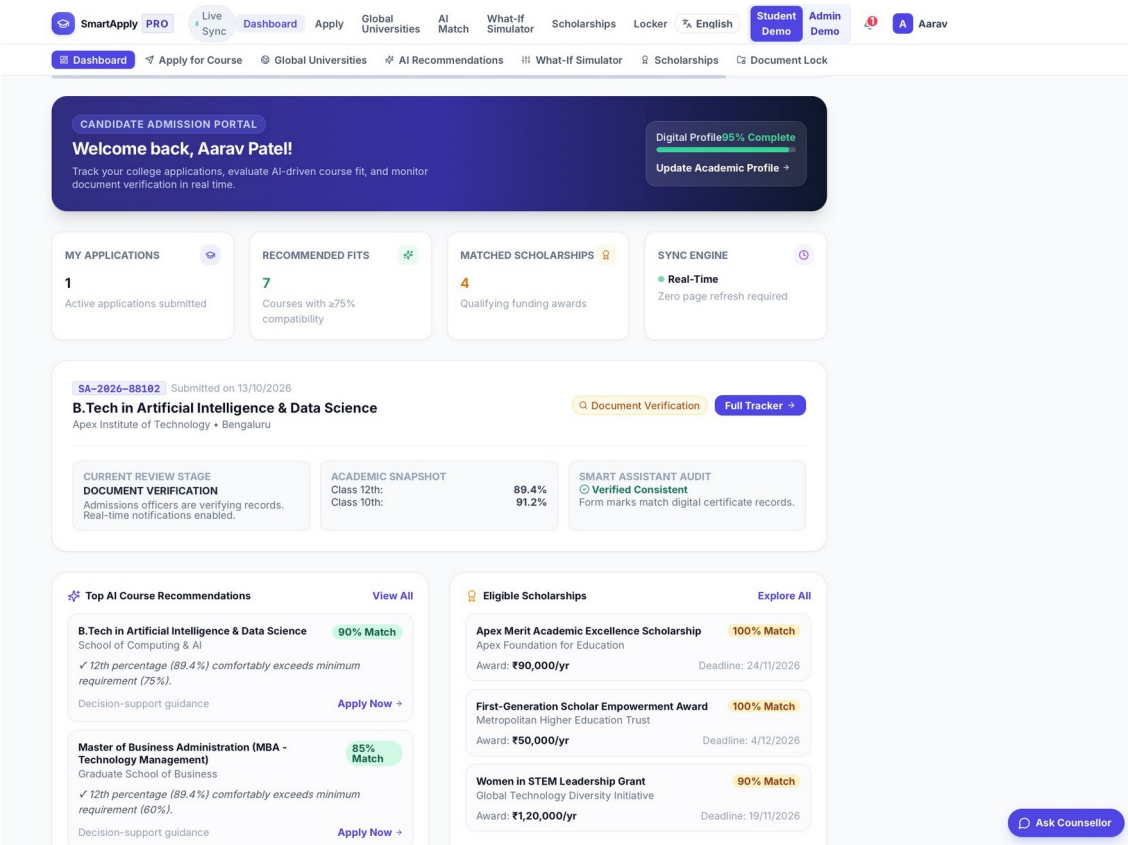


Figure A.3 Student dashboard with live application status

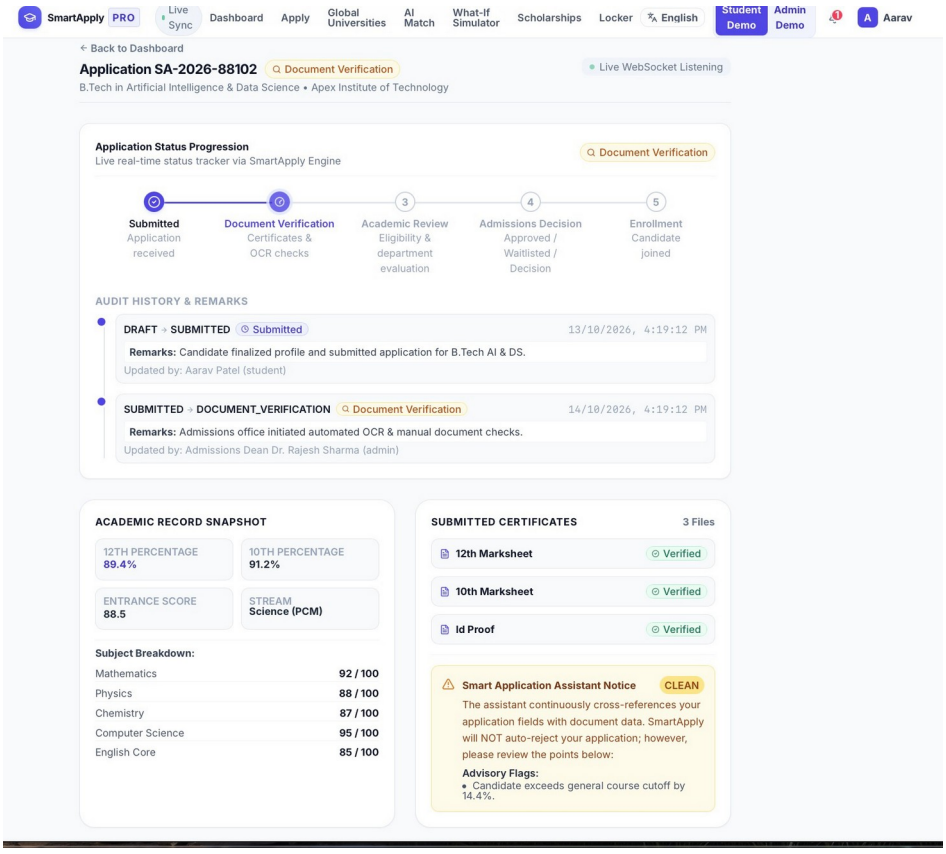


Figure A.4 Application tracker and status timeline

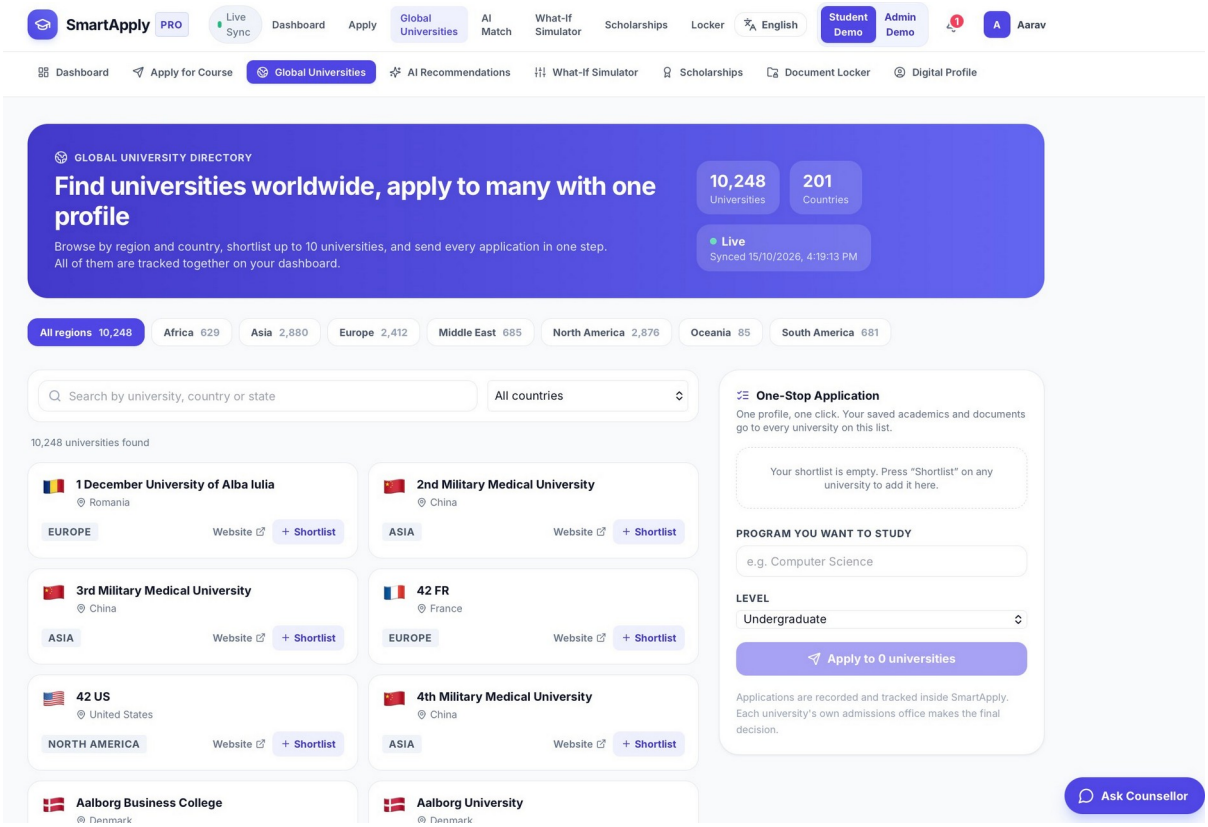


Figure A.5 Global university directory with region filter and shortlist

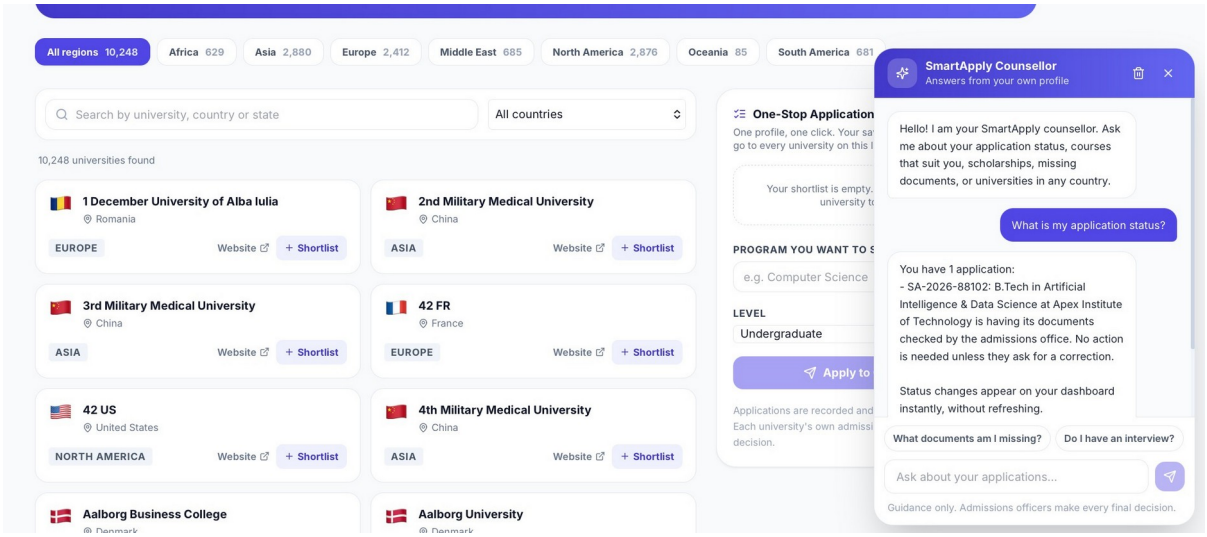


Figure A.6 Counsellor chatbot answering an application status question

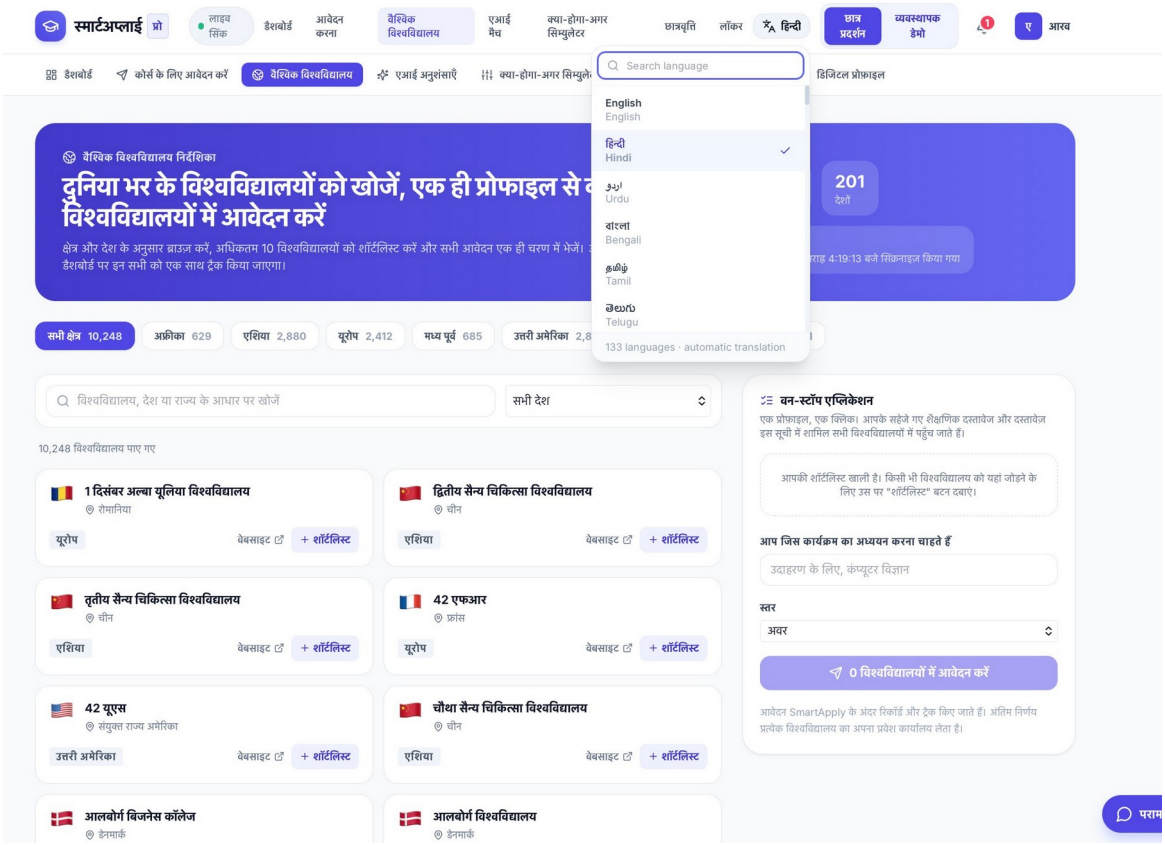


Figure A.7 Language switcher with the page shown in another language

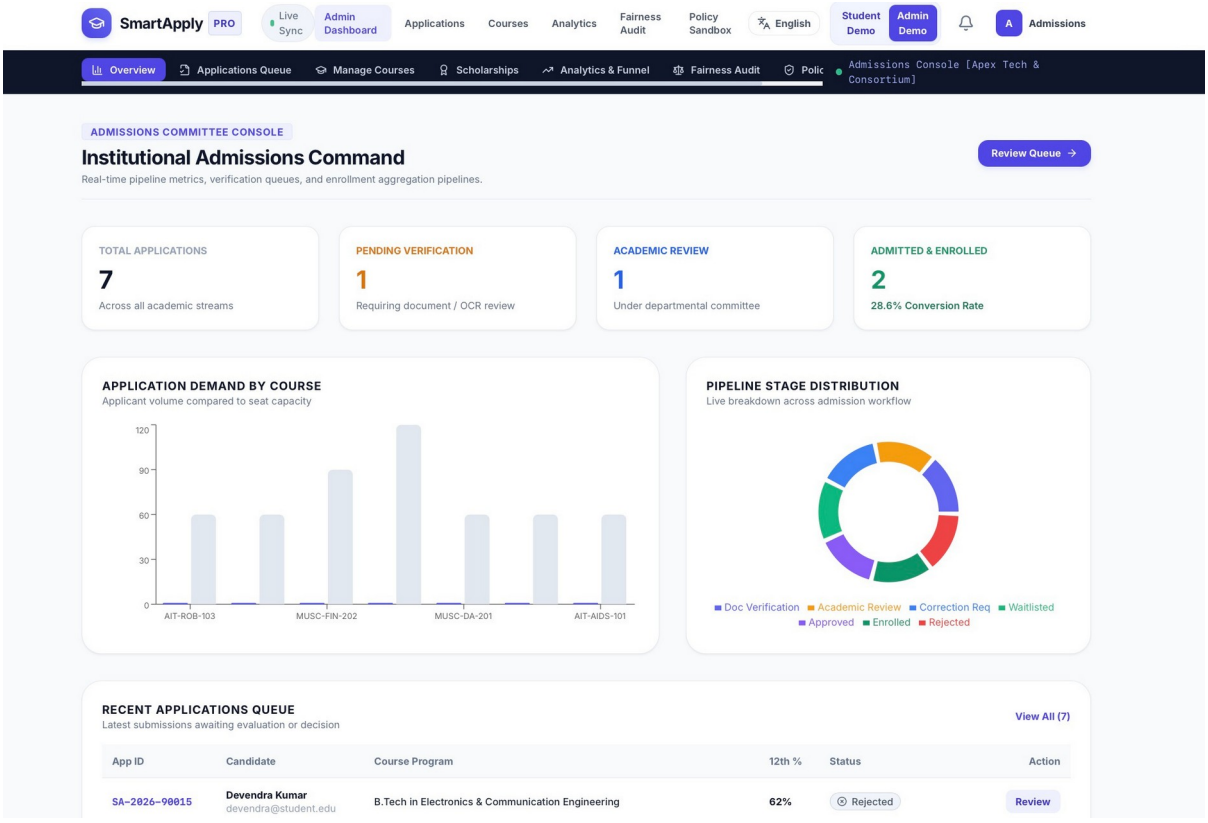


Figure A.8 Admin dashboard with pipeline and course demand charts

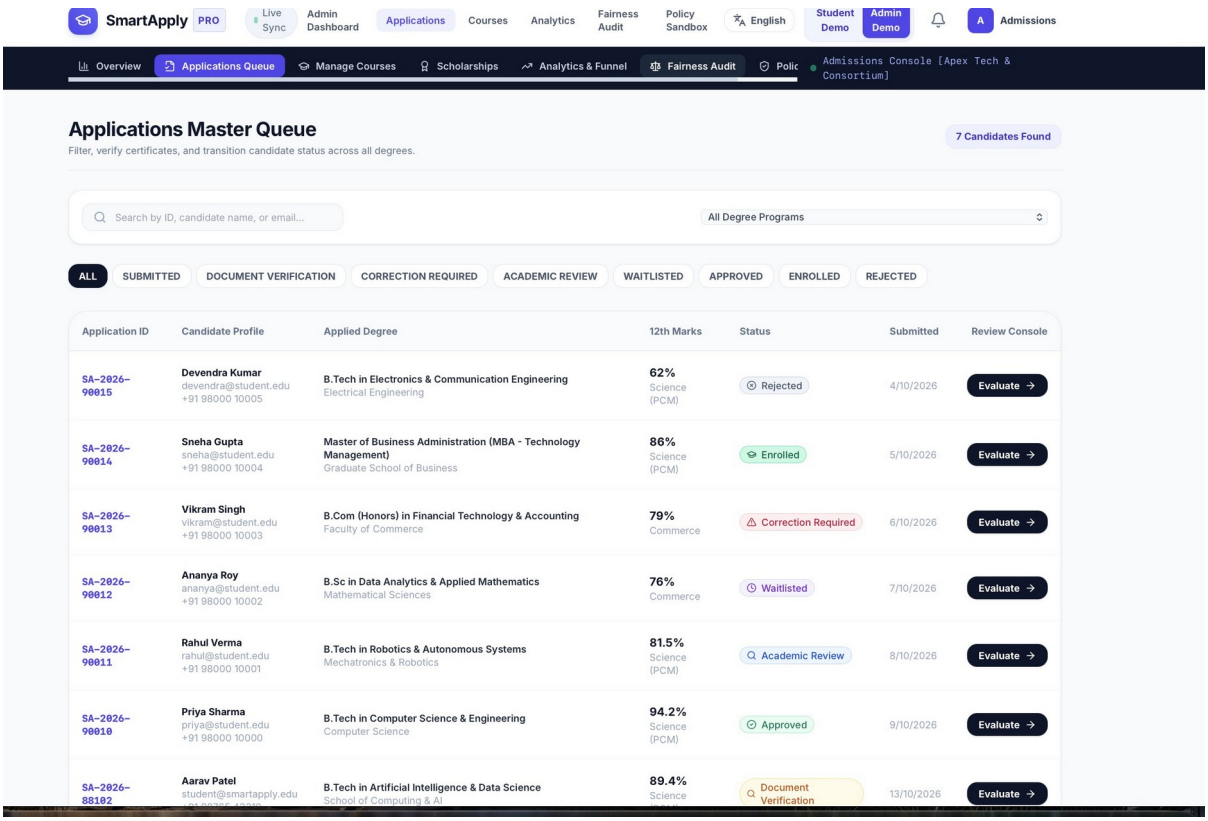


Figure A.9 Admin application review and status transition

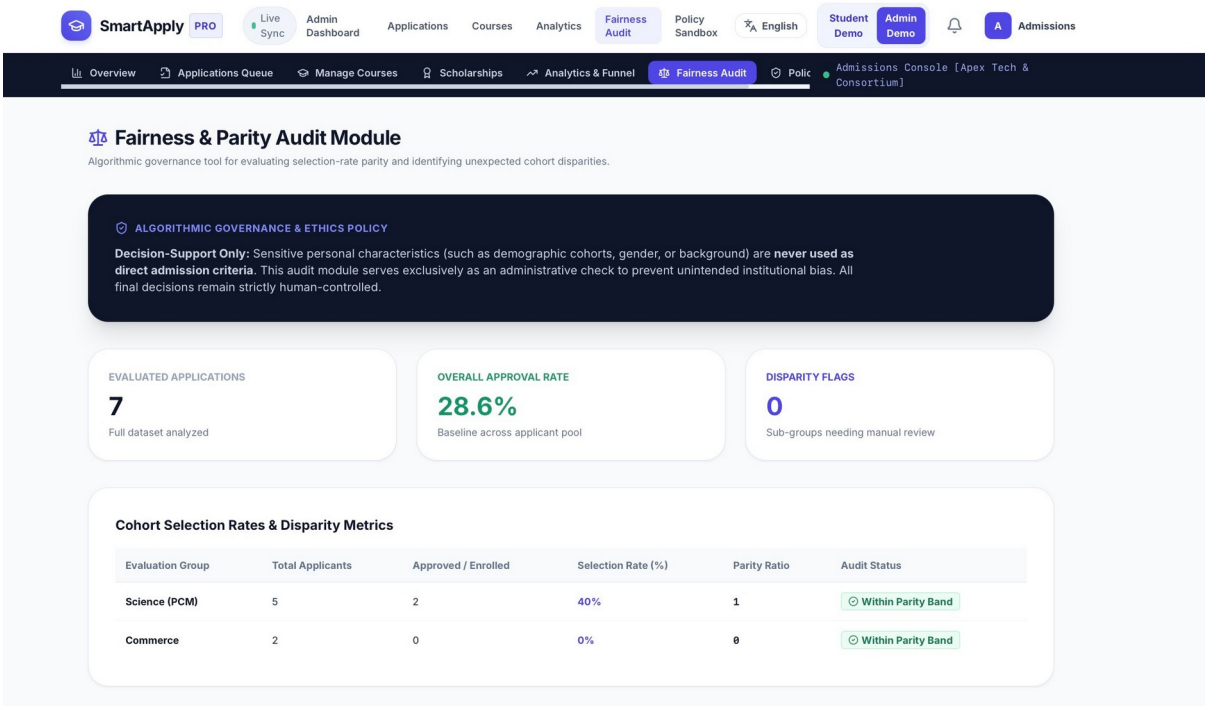


Figure A.10 Fairness audit report

REFERENCES

[1] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000.

[2] MongoDB, Inc., “MongoDB Manual.” [Online]. Available: https://www.mongodb.com/docs/manual/

[3] OpenJS Foundation, “Express – Node.js web application framework.” [Online]. Available: https://expressjs.com/

[4] Meta Platforms, Inc., “React – The library for web and native user interfaces.” [Online]. Available: https://react.dev/

[5] OpenJS Foundation, “Node.js Documentation.” [Online]. Available: https://nodejs.org/docs/

[6] Automattic, “Mongoose ODM Documentation.” [Online]. Available: https://mongoosejs.com/docs/

[7] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, Internet Engineering Task Force, Dec. 2011.

[8] Socket.IO, “Socket.IO Documentation.” [Online]. Available: https://socket.io/docs/v4/

[9] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015.

[10] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in Proc. USENIX Annual Technical Conference, FREENIX Track, 1999, pp. 81–91.

[11] U.S. Equal Employment Opportunity Commission, “Uniform Guidelines on Employee Selection Procedures,” 29 C.F.R. Part 1607, 1978.

[12] Hipo, “University Domains List.” [Online]. Available: https://github.com/Hipo/university-domains-list

[13] Tailwind Labs, “Tailwind CSS Documentation.” [Online]. Available: https://tailwindcss.com/docs

[14] Vite, “Vite – Next Generation Frontend Tooling.” [Online]. Available: https://vite.dev/

[15] Mozilla, “MDN Web Docs – HTTP.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP